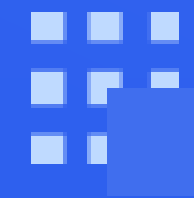




软件测试

引导性随机测试

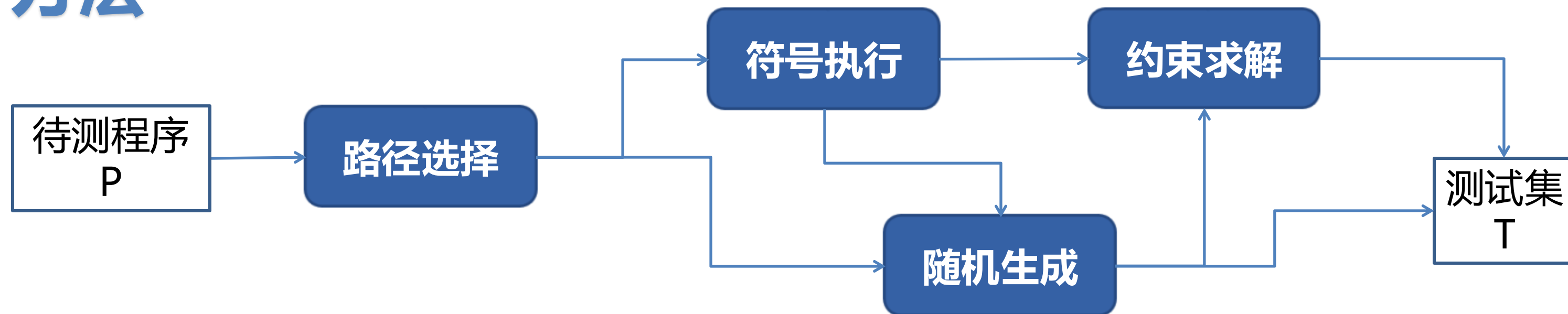


方法概述



总体目标：选择适当的测试生成方法，快速生成测试覆盖程序的各种路径或指定路径。

方法



引导性随机测试

定义	目标	优点
随机抽样结合符号执行反馈信息筛选获得测试集 T 。	利用符号执行的路径探索能力优化随机测试的效率。	提高测试覆盖率并期待发现深层次的程序缺陷。

符号执行(Symbolic Execution)

定义:

一种静态测试方法，通过构造符号化输入代替具体输入，探索程序路径并生成路径约束条件，最后求解这些约束条件得到程序的测试输入范围。

流程:

1. 符号化变量：使用符号值代替数字值来表示程序变量。
2. 模拟程序执行：不执行实际程序，基于中间语言模拟程序执行。
3. 记录路径条件：记录执行过程中经过的语句序列，即变量符号值的约束。
4. 求解约束条件：使用约束求解器得到满足路径条件的测试输入。
5. 生成测试用例：将求解结果转化为测试用例格式。

狭义符号执行

广义符号执行

符号执行(Symbolic Execution)

```
int main() {  
    int x, y;  
    scanf("%d %d", &x, &y);  
    x=x+2;  
    y=y+x;  
    if (x > 5) {  
        printf("y = %d\n", y);  
        if (y > 0){  
            printf("x = %d\n", x);  
        }  
    }  
    return 0;  
}
```

符号执行:

1. 符号化变量: 将 x 和 y 视为符号变量 (或表示为 $x = a$, $y = b$) 。
2. 模拟执行与记录路径条件:
 - 首先执行 `scanf` 语句, 这里我们不考虑实际的输入, 而是将 x 和 y 作为符号值继续执行。
 - 执行赋值语句
 - 变量 x 替换为 $x+2$;
 - 变量 y 替换为 $y+x+2$ 。
 - 第一个 `if` 语句, 记录下路径条件。如果走 $x > 5$ 这个分支, 也就是满足路径条件 $PC1: x+2 > 5$, 则执行输出 $y + x + 2$;
 - 第二个 `if` 语句, 记录下路径条件。路径条件就是 $PC2: x+2 > 5 \ \&\& \ y+x+2 > 0$, 执行输出 $x + 2$ 。

符号执行(Symbolic Execution)

```
int main() {  
    int x, y;  
    scanf("%d %d", &x, &y);  
    x=x+2;  
    y=y+x;  
    if (x > 5) {  
        printf("y = %d\n", y);  
        if (y > 0){  
            printf("x = %d\n", x);  
        }  
    }  
    return 0;  
}
```

符号执行:

1. 符号化变量: 将 x 和 y 视为符号变量 (或表示为 $x = a$, $y = b$) 。
2. 模拟执行与记录路径条件:
 - 首先执行 `scanf` 语句, 这里我们不考虑实际的输入, 而是将 x 和 y 作为符号值继续执行。
 - 执行赋值语句
 - 变量 x 替换为 $x+2$;
 - 变量 y 替换为 $y+x+2$ 。
 - 第一个 `if` 语句, 记录下路径条件。如果走 $x > 5$ 这个分支, 也就是满足路径条件 $PC1: x+2 > 5$, 则执行输出 $y + x + 2$;
 - 第二个 `if` 语句, 记录下路径条件。路径条件就是 $PC2: x+2 > 5 \&\& y+x+2 > 0$, 执行输出 $x + 2$ 。

约束求解:

1. 求解 $PC1: x+2 > 5$, 即 $x > 3$, 例如求解或随机得到 $x=7$; 不满足 $PC1$ 的解为 $x \leq 3$, 例如 $x=0$ 。
2. 求解 $PC2: x+2 > 5 \&\& y+x+2 > 0$, 即 $x > 3 \&\& y+x > -2$, 例如求解或随机得到 $x=4, y=0$; 不满足 $PC2$ 的解为 $x=4, y=-2$ 。

符号执行的路径爆炸问题：

随着程序规模的增大和分支结构的增多，程序的执行路径数量会呈指数级增长。一个包含 n 个独立if - else分支的程序，理论上有 2^n 条不同的执行路径。这使得符号执行在分析大型程序时，需要处理海量的路径，常常因为资源耗尽而无法完成分析。

- 路径剪枝：通过静态分析技术提前识别出不可行路径，将其从分析范围中排除。
- 启发式搜索：采用启发式算法对路径优先分析那些更有可能发现问题的路径。
- 抽象解释：结合抽象解释对程序状态进行抽象表示，减少符号执行需要处理的状态数量。

符号执行的路径爆炸问题:

```
def pathpruning(x, y):  
    if x > 5:  
        result = x+1  
    else:  
        result = x+2  
        if y > 10:  
            result = y+1  
        else:  
            result = y+2  
        if x + y > 20:  
            result = x+y  
        else:  
            result = x-y  
        .....  
    return result
```

这个程序具有三个分支, 每个有两个选择。

第一个 if 语句, 产生两条路径分支:

A1: $x > 5$

A0: $x \leq 5$

第二个 if 语句, 产生两条路径分支:

B1: $y > 10$

B0: $y \leq 10$

第三个 if 语句, 产生两条路径分支:

C1: $x+y > 20$

C0: $x+y \leq 20$

通过分析可知A0&B0&C1是不可满足的, 因为 $x \leq 5$ 且 $y \leq 10$ 时,
 $x+y$ 不会大于 20, 可以对该路径剪枝, 减少后期分析成本。

思考: 路径剪枝与极小不可满足和极大可满足的关系

符号执行的内部结构问题：

指针和引用使得程序中的数据关系变得复杂，符号执行需要准确跟踪指针的指向和引用的对象，以正确分析程序的行为。否则，可能会导致错误的路径探索或遗漏某些路径。

```
int main() {  
    int a = 10;  
    int *p = &a;  
    int **pp = &p;  
  
    if (*pp != NULL) {  
        **pp = 20;  
    }  
    printf("%d\n", a);  
    return 0;  
}
```

在这段代码中，存在指针p指向变量a，以及二级指针pp指向指针p。符号执行时需要准确跟踪pp、p和a之间的关系，以确定if条件的真假以及对a的赋值操作。如果不能正确处理指针的层级关系，就可能无法正确分析程序的行为。

符号执行的内部结构问题：

数组的索引常常导致符号执行需要处理大量的可能索引值，增加了路径数量和分析的复杂性。多维数组更是进一步加剧了这种复杂性，因为需要同时处理多个维度的索引。

```
public class ArrayExample {  
    public static void main(String[] args) {  
        int[] array = {1, 2, 3, 4, 5};  
        int index = 2;  
        if (index >= 0 && index < array.length) {  
            array[index] = 10;  
        }  
        System.out.println(array[2]);  
    }  
}
```

这里的索引index可以是一个符号变量，符号执行需要考虑index在满足 $\text{index} \geq 0 \ \&\& \ \text{index} < \text{array.length}$ 条件下的各种可能取值，以及对数组元素的赋值操作。如果是多维数组，例如`int[][] matrix = new int[3][3];`，处理起来会更加复杂，需要同时考虑行和列的索引变化对数组元素的影响。

符号执行的内部结构问题：

- 结构复杂性：符号执行时需要跟踪每个节点的指针关系，以确定链表的结构和遍历路径。由于链表的长度和结构可能在运行时动态变化，符号执行需要处理各种可能的情况，这增加了分析的复杂性。
- 结构多样性：树结构具有多种类型，如二叉树、多叉树等，且树的深度和节点数量可能在运行时动态变化。符号执行需要处理不同类型树的遍历、插入、删除等操作，同时要考虑树的平衡性、节点的层次关系等因素。
- 哈希冲突处理：哈希表通过哈希函数将键值映射到数组的索引位置。然而，不同的键可能会产生相同的哈希值，导致哈希冲突。符号执行需要处理哈希冲突的情况，确保在插入、查找和删除操作时能够正确访问哈希表中的元素。

符号执行的外部函数问题：

当程序中存在外部函数调用时，符号执行很难对这些外部函数的行为进行准确建模。如操作系统、数据库等，导致符号执行无法准确地模拟程序的执行过程，影响分析结果的准确性。

- 函数摘要：为外部函数生成函数摘要，描述函数的输入输出关系。
- 模拟执行：对于一些常见的外部函数，可以编写模拟代码来模拟其行为。
- 动态信息结合：结合动态执行的信息来辅助处理外部函数调用。可以在程序运行时收集外部函数的输入输出信息，然后将这些信息用于符号执行过程中，提高分析的准确性。

符号执行的外部函数问题：

库函数通常以二进制形式提供，符号执行难以获取其内部实现细节，只能将其视为黑盒。符号执行无法准确分析库函数调用对程序状态的影响，从而难以确定程序的后续执行路径。

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int random_num = rand();
    if (random_num > 100) {
        printf("随机数大于100\n");
    }
    else {
        printf("随机数小于等于100\n");
    }
    return 0;
}
```

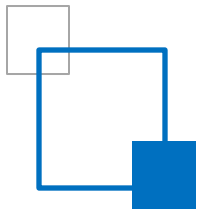
在符号执行时，rand()函数是一个黑盒，符号执行工具无法得知它会返回什么值。这就导致符号执行需要考虑rand()函数所有可能的返回值，极大地增加了需要探索的路径数量。若要精确分析程序行为，可能需要额外的信息或对rand()函数的模拟，但这并非总能轻易实现。

符号执行的约束求解问题：

某些复杂的约束条件可能会超出约束求解器的能力范围，导致求解时间过长或者无法求解。

例如，包含非线性算术运算、量词的约束条件，对于现有的约束求解器来说是比较困难的。

- 简化约束条件：在符号执行过程中，对路径条件进行预处理，尽量简化约束。例如，利用代数化简、逻辑等价变换等方法，将复杂的约束转化为更容易求解的形式。
- 使用近似求解：对于一些难以精确求解的约束条件，可以采用近似求解的方法，虽然不能保证找到所有解，但可以在可接受的时间内得到一些有用的结果。
- 多求解器结合：不同的约束求解器在处理不同类型的约束条件时具有不同的优势，可以将多个求解器结合起来，根据约束条件的特点选择合适的求解器进行求解。



静态符号执行与动态符号执行

引导性随机测试

内容	静态符号执行	动态符号执行
运行原理	为每个执行路径生成对应的符号约束，不依赖于程序实际执行，全面对所有可能路径符号化分析。	结合了动态执行和符号执行，先对程序进行一次或多次动态执行，记录实际执行路径和变量取值，然后利用这些信息引导符号执行，优先探索与实际执行路径相似的路径。
路径探索	尝试穷举程序的所有可能执行路径，路径数量可能呈指数级增长。	借助动态执行的信息，减少不必要的路径探索，优先关注与实际执行路径相关的路径，从而降低路径搜索空间。
约束求解	对每条路径都需要求解符号约束，以确定该路径是否可行，约束求解次数较多。	可以利用动态执行中得到的变量取值来简化约束求解过程，缩小符号约束的求解空间。
执行效率	由于要探索所有路径，时间和空间开销大，效率较低，但可以发现程序中更多潜在故障。	通过减少路径探索，提高了效率。依赖于动态执行的信息，可能会遗漏一些未在动态执行中出现的路径，故障检测率相对传统符号执行可能较低。

动态符号执行示例

```
int f( int x) {  
    return 2 * x;  
}  
  
int h(int x, int y) {  
    if (x != y) {  
        if (f(x) == x + 10) {  
            abort(); /* error */  
        }  
    }  
}
```

路径分析:

- 路径分支1: $x == y \rightarrow$ 不需要测试。
- 路径分支2: $x \neq y$
 - 子路径 2a:
 $f(x) == x + 10$
测试输入 $x=5, y=6$
 - 子路径 2b:
 $f(x) \neq x + 10$
测试输入 $x=4, y=6$

如何求解获得
测试输入值?

动态符号执行示例

```
int f( int x) {  
      
}  
  
int h(int x, int y) {  
    if (x != y) {  
        if (f(x) == x + 10) {  
            abort(); /* error */  
        }  
    }  
}
```

路径分析:

- 路径1: $x == y \rightarrow$ 不需要测试。
- 路径2: $x \neq y$
 - 子路径 2a:
 $f(x) == x + 10$
测试输入 $x=5, y=6$
 - 子路径 2b:
 $f(x) \neq x + 10$
测试输入 $x=4, y=6$

如何求解获得
测试输入值?

符号执行示例-均值方差MeanVar程序



代码逻辑: 输入数据变量 X 和 长度 L : 约束条件: $L > 0$

路径分析:

1. 第一个 for 循环:
约束条件: $L > 0$ 。
2. 计算 $\text{sum} = a[0] + a[1] + \dots + a[L-1]$ 。
3. 第二个 for 循环:
计算 $\text{varsum} = \sum_{i=0}^{L-1} (a[i] - m)^2$
4. 其中 m 是均值。
5. 输出结果: $\text{mean} = \frac{\text{sum}}{L}$ 和 $\text{var} = \frac{\text{varsum}}{L-1}$

符号执行挑战?
如何解决挑战?

回顾与讨论

采用随机或目标
引导符号执行动
态测试

- 静态符号执行对程序的所有路径进行系统性探索，不依赖于特定的输入种子和程序具体执行，有助于发现隐藏在罕见路径中的程序漏洞。
- 动态符号执行减少了路径探索的数量和约束求解的复杂度，面对复杂的大型程序能够在合理的时间和资源消耗内完成分析。

- 动静态结合是测试优化的基本思路。
- 执行实时反馈更有效引导符号执行。
- 随机是固定策略无效时的有益补充。